



Anno scolastico 2025/2026 I3A

MODULO 306

Elisa Negri, Julie Jane Gambale

SPAI LOCARNO | SISA, Cornér banca

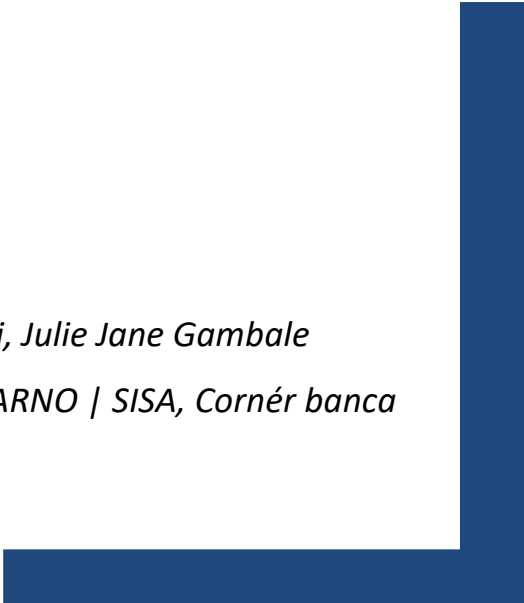


Table of Contents

Scheda progetto	Error! Bookmark not defined.
1. Mandato del progetto	3
Contesto	3
Obiettivi formativi	3
Modalità di svolgimento	3
2. Scelta del progetto	4
Valore didattico	4
3. Obiettivi del progetto	5
Obiettivi principali	5
Requisiti funzionali	5
Requisiti non funzionali	5
4. Metodo di lavoro	6
Fase di avvio	6
Fase di concezione	6
Fase di realizzazione	6
Fase di chiusura	6
5. Pianificazione del progetto	7
Suddivisione delle attività	7
Sequenza di lavoro	7
Attività principali e risultato atteso	7
6. Rischi tecnici	8
Rischi principali	8
Mitigazioni possibili	8
7. Tecnologie utilizzate	9
Pacchetti principali	9
Dipendenze in pubspec.yaml	9
8. Architettura del progetto	10
Struttura dei file	10
Flusso generale	10
9. Avvio e configurazione	11

10. Home e navigazione	12
11. Timer Pomodoro	13
12. Gestione task	14
13. Generazione delle flashcard tramite API.....	15
14. Visualizzazione delle flashcard	16
15. Interfaccia grafica	17
Scelte UI.....	17
16. Persistenza e dati.....	17
Limiti attuali	17
17. Test e verifiche	18
Test consigliati	18
Comandi principali.....	18
18. Miglioramenti futuri e conclusione	18
Miglioramenti futuri.....	18
Conclusione	19

1. Mandato del progetto

Contesto

Il modulo si inserisce nel percorso formativo degli apprendisti informatici al terzo anno, ramo sviluppo applicativo. L'obiettivo è realizzare un applicativo completo, dalla pianificazione fino alla presentazione finale, in un contesto che riproduce dinamiche reali di lavoro.

StudySpace risponde a questo mandato con una app mobile pensata per aiutare studenti e apprendisti a organizzare lo studio. Il progetto include una parte funzionale semplice, come timer e task, e una parte più tecnica legata alla generazione automatica di flashcard tramite una API locale.

Obiettivi formativi

- pianificare lo sviluppo di un progetto software in maniera strutturata
- applicare metodologie di progettazione e sviluppo con tecnologie scelte in autonomia
- produrre documentazione tecnica chiara, completa e coerente
- presentare il progetto a un pubblico tecnico e non tecnico
- dimostrare capacità di integrazione tra interfaccia, logica applicativa e servizio esterno

Modalità di svolgimento

- durata del modulo: 36 unità didattiche distribuite lungo l'anno scolastico
- tema e tecnologie: scelti dal gruppo e verificati con il docente
- produzione attesa: applicativo funzionante con documentazione tecnica e di progetto
- presentazione finale: spiegazione del processo, delle scelte tecniche e delle difficoltà incontrate

2. Scelta del progetto

Abbiamo deciso di creare una app che faccia da sostegno allo studio agli studenti. L'idea nasce da un bisogno concreto: durante lo studio bisogna spesso gestire tempo, compiti, pause e ripasso. Avere questi strumenti in un'unica applicazione rende il lavoro più organizzato.

L'applicazione contiene un timer per gestire momenti di studio e pause, una sezione per i task dello studente e una schermata che permette all'utente di inserire un argomento da trasformare in flashcard.

La generazione delle flashcard avviene tramite una chiamata alla API locale di Ollama. La risposta è un JSON che contiene domande e risposte; l'app lo legge e lo usa per popolare la schermata delle flashcard.

Questo progetto è un buon compromesso: la base dell'app è semplice e comprensibile, ma l'aggiunta della generazione automatica di flashcard tramite AI introduce una complessità tecnica interessante.

Valore didattico

- utilizzo di Flutter e Dart in un progetto mobile reale
- organizzazione del codice in pagine e widget
- gestione dello stato con StatefulWidget e setState
- uso di callback per passare dati tra componenti
- chiamata HTTP asincrona verso un servizio esterno
- parsing JSON e salvataggio locale di un file

3. Obiettivi del progetto

Obiettivi principali

- creare una app mobile con struttura chiara e modulare
- costruire una interfaccia semplice, adatta a studenti
- gestire navigazione interna tra strumenti diversi
- sviluppare un timer Pomodoro per sessioni focus
- sviluppare una lista di task con priorità
- integrare una API esterna o locale per generare contenuti di studio
- trasformare la risposta JSON dell'AI in flashcard navigabili
- documentare le parti principali del codice con esempi e spiegazioni

Requisiti funzionali

- l'utente deve poter aprire l'app e riconoscere subito le sezioni principali
- l'utente deve poter usare un timer focus con start, stop, reset e durata modificabile
- l'utente deve poter creare task con priorità Low, Medium o High
- l'utente deve poter generare flashcard partendo da un tema scritto
- l'utente deve poter sfogliare le flashcard avanti e indietro

Requisiti non funzionali

- il codice deve rimanere leggibile e diviso in file coerenti
- le dipendenze devono essere dichiarate in pubspec.yaml
- l'interfaccia deve rimanere coerente nel tema grafico
- la chiamata API deve usare JSON e un formato prevedibile
- la documentazione deve spiegare sia il progetto sia il codice

4. Metodo di lavoro

Il lavoro può essere letto secondo una logica simile al metodo Hermes. Questo metodo aiuta a separare le fasi di progetto e a non confondere analisi, progettazione, realizzazione e chiusura.

Fase di avvio

Nella fase di avvio è stata definita l'idea di progetto: una app di supporto allo studio. Sono stati individuati gli obiettivi principali e i rischi iniziali, in particolare la complessità della parte API e la necessità di mantenere il codice ordinato.

Fase di concezione

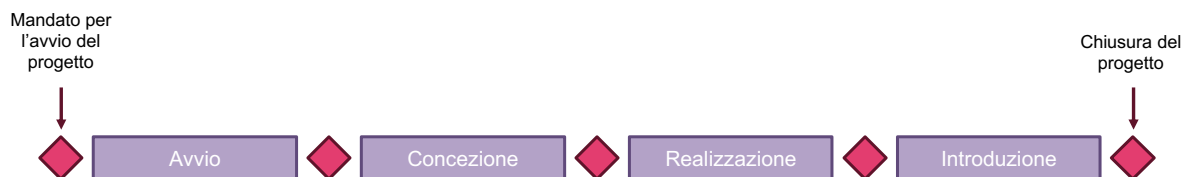
Nella fase di concezione sono state definite le schermate principali: Pomodoro, Tasks, generazione flashcard e visualizzazione flashcard. È stata scelta una struttura basata su pagine e widget, in modo da separare navigazione, UI e logica.

Fase di realizzazione

Nella fase di realizzazione sono stati sviluppati i componenti Flutter: StudyApp, StudyHomePage, NavigationCards, PomodoroTimerCard, TaskPage, ApiAccessCard e FlashcardCard.

Fase di chiusura

La chiusura comprende test manuali, sistemazione della documentazione e preparazione della presentazione finale. Questa documentazione finale nasce proprio per consolidare tutte le informazioni in un unico file.



5. Pianificazione del progetto

Suddivisione delle attività

- analisi dell'idea e definizione del target
- setup del progetto Flutter e configurazione delle dipendenze
- creazione della struttura lib/app, lib/pages e lib/widgets
- sviluppo della schermata principale con header e navigazione
- implementazione del timer Pomodoro
- implementazione della gestione task
- integrazione API per generare flashcard
- visualizzazione e navigazione delle flashcard generate
- test manuali, documentazione e preparazione della presentazione

Sequenza di lavoro

1. Creare il progetto Flutter e verificare l'avvio su emulatore o dispositivo.
2. Impostare StudyApp e StudyHomePage come base dell'applicazione.
3. Aggiungere navigazione e sezioni principali.
4. Sviluppare il timer Pomodoro e verificare start, stop e reset.
5. Sviluppare la task list e verificare inserimento, completamento ed eliminazione.
6. Integrare Ollama tramite chiamata HTTP e decodificare la risposta JSON.
7. Passare la lista di flashcard alla schermata di visualizzazione.
8. Rifinire UI, testare i flussi e completare la documentazione.

Attività principali e risultato atteso

- Analisi idea: definizione degli obiettivi e delle funzioni.
- Setup Flutter: progetto avviabile con dipendenze installate.
- UI principale: home, header e navigazione funzionanti.
- Funzioni studio: timer e task list utilizzabili.
- AI flashcard: chiamata API, JSON e visualizzazione delle card.
- Test e documentazione: relazione finale e supporto alla presentazione.

6. Rischi tecnici

I rischi tecnici più importanti riguardano soprattutto la generazione delle flashcard, la gestione dello stato e la persistenza dei dati. Il progetto è funzionante, ma alcune parti possono essere rese più robuste.

Rischi principali

- Ollama deve essere attivo localmente su localhost:11434, altrimenti la generazione non funziona.
- La risposta dell'AI deve essere JSON valido; se il formato cambia, jsonDecode può fallire.
- Le flashcard vengono salvate in un file locale, ma non vengono ancora ricaricate automaticamente all'avvio.
- Task e sessioni Pomodoro sono mantenuti in memoria e si perdono alla chiusura dell'app.
- Su schermi piccoli alcuni contenuti potrebbero richiedere adattamenti responsive.
- La logica API è ancora dentro il widget; in futuro sarebbe meglio separarla in un service dedicato.

Mitigazioni possibili

- controllare lo stato del servizio Ollama prima della chiamata
- aggiungere messaggi di errore visibili all'utente
- validare meglio il JSON ricevuto
- salvare e ricaricare task e flashcard
- aggiungere test automatici per parsing JSON e timer

7. Tecnologie utilizzate

Abbiamo deciso di sviluppare questa applicazione usando Flutter e Dart. Flutter è un SDK open source creato da Google che permette di costruire interfacce multiplatforma con una sola codebase.

Dart è il linguaggio usato da Flutter. È fortemente tipizzato e permette di scrivere widget, gestire eventi e aggiornare l'interfaccia tramite setState. Durante lo sviluppo è molto utile anche l'hot reload, perché permette di vedere rapidamente le modifiche.

Pacchetti principali

- flutter: framework principale dell'applicazione
- cupertino_icons: icone in stile iOS usate in alcuni pulsanti
- http: pacchetto usato per inviare la richiesta POST a Ollama
- path_provider: pacchetto usato per trovare una cartella locale dove salvare flashcards.json
- flash_card: pacchetto usato per visualizzare carte fronte/retro

Dipendenze in pubspec.yaml

```
dependencies:  
  flash_card: "^0.0.5"  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.8  
  http: ^1.6.0  
  path_provider: ^2.1.0
```

8. Architettura del progetto

Il codice è diviso in modo semplice: le pagine decidono quale sezione mostrare, mentre i widget contengono la grafica e la logica dei singoli strumenti. Questa divisione rende il progetto più facile da leggere e da estendere.

Struttura dei file

- lib/main.dart: punto di ingresso dell'applicazione.
- lib/app/studyApp.dart: configurazione MaterialApp, tema e pagina iniziale.
- lib/pages/: schermate principali che collegano i widget.
- lib/widgets/: componenti visuali e logici riutilizzabili.
- pubspec.yaml: dipendenze e configurazione Flutter.
- android/ e ios/: configurazioni native generate da Flutter.

Flusso generale

1. main.dart avvia StudyApp con runApp.
2. StudyApp crea MaterialApp e apre StudyHomePage.
3. StudyHomePage mostra header, navigazione e pagina selezionata.
4. ApiAccessCard genera flashcard e le passa alla home tramite callback.
5. FlashcardPage riceve la lista e la mostra dentro FlashcardCard.

9. Avvio e configurazione

Il file `main.dart` è il punto di ingresso. Flutter esegue `runApp` e monta il widget principale `StudyApp`.

```
import 'package:flutter/material.dart';
import 'app/studyApp.dart';

void main() {
  runApp(const StudyApp());
}
```

`StudyApp` definisce il tema Material, il titolo dell'app e la pagina iniziale. In questo modo la configurazione generale rimane separata dalle schermate operative.

```
class StudyApp extends StatelessWidget {
  const StudyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'StudySpace',
      theme: ThemeData(
        primarySwatch: Colors.deepPurple,
        primaryColor: Colors.deepPurple,
        fontFamily: 'Montserrat',
        useMaterial3: true,
      ),
      home: const StudyHomePage(),
    );
  }
}
```

10. Home e navigazione

StudyHomePage è il centro dell'app. Mantiene l'indice della pagina selezionata e la lista delle flashcard generate. Questa scelta rende semplice passare dati dalla schermata API alla schermata flashcard.

```
int selectedIndex = 0;
List _flashcards = [];

void handleGoFlashcards(List flashcards) {
  setState(() {
    _flashcards = flashcards;
    selectedIndex = 3;
  });
}

void handleSelected(int index) {
  setState(() {
    selectedIndex = index;
  });
}
```

IndexedStack mantiene le pagine nella stessa posizione e mostra solo quella selezionata. Questo evita una navigazione complessa e rende immediato il cambio tra Pomodoro, task, generazione e flashcard.

```
IndexedStack(
  index: selectedIndex,
  children: pages,
)
```

11. Timer Pomodoro

PomodoroTimerCard è uno StatefulWidget perché deve aggiornare il tempo ogni secondo. Lo stato contiene timer, secondi rimanenti, durata totale, sessioni completate e flag di esecuzione.

```
Timer? _timer;  
int _remainingSeconds = 600;  
late int _totalSeconds = 600;  
int _sessionsCompleted = 0;  
bool _isRunning = false;
```

La funzione startTimer crea un Timer.periodic. Ogni secondo il valore dei secondi rimanenti diminuisce. Quando arriva a zero, il timer viene cancellato e il contatore delle sessioni aumenta.

```
void startTimer() {  
  if (_isRunning) return;  
  
  _isRunning = true;  
  _timer = Timer.periodic(const Duration(seconds: 1), (timer) {  
    setState(() {  
      if (_remainingSeconds > 0) {  
        _remainingSeconds--;  
      } else {  
        timer.cancel();  
        _isRunning = false;  
        _sessionsCompleted++;  
      }  
    });  
  });  
}
```

12. Gestione task

La pagina task definisce un piccolo modello MyTask con titolo, priorità e stato di completamento. La lista viene aggiornata con setState quando l'utente aggiunge, completa o elimina un task.

```
class MyTask {  
  String title;  
  String priority;  
  bool isDone;  
  
  MyTask({  
    required this.title,  
    required this.priority,  
    this.isDone = false,  
  });  
}
```

TaskPage mantiene la lista in memoria. Quando l'utente scrive un titolo e preme il pulsante aggiungi, viene creato un nuovo MyTask e la UI viene aggiornata.

```
if (_controller.text.isNotEmpty) {  
  setState(() {  
    _myTasks.add(  
      MyTask(  
        title: _controller.text,  
        priority: _selectedPriority,  
      ),  
    );  
    _controller.clear();  
  });  
}
```

Il widget taskCard riceve callback per completare o cancellare il task. Questo mantiene la UI della singola card separata dalla lista dati gestita da TaskPage.

13. Generazione delle flashcard tramite API

ApiAccessCard legge il tema inserito dall'utente, invia una richiesta POST a Ollama e chiede al modello di rispondere solo con JSON valido. Dopo la risposta, il contenuto viene pulito, salvato in flashcards.json e convertito in una lista.

```
final response = await http.post(
  Uri.parse('http://localhost:11434/api/chat'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({
    "model": "gemma3n",
    "stream": false,
    "messages": [
      {
        "role": "system",
        "content": "You are an assistant that generates educational flashcards."
      },
      {"role": "user", "content": "Genera flashcard sul tema: $tema"}
    ]
  }),
);
```

Questa parte mostra il passaggio da testo grezzo a dati usabili dall'app. decoded["flashcards"] viene passato alla callback goFlashcards, che nella home aggiorna lo stato e apre la pagina delle flashcard.

```
final data = jsonDecode(response.body);
String content = data['message']['content']
  .replaceAll('```json', '')
  .replaceAll('```', '')
  .trim();

final decoded = jsonDecode(content);
widget.goFlashcards(decoded["flashcards"] as List);
```


14. Visualizzazione delle flashcard

FlashcardCard riceve la lista e usa `currentIndex` per sapere quale carta mostrare. Se la lista è vuota restituisce `SizeBox.shrink`, evitando errori di accesso a un indice inesistente.

```
if (flashcards.isEmpty) {  
  return const SizeBox.shrink();  
}
```

Quando cambia la lista, `didUpdateWidget` rimette `currentIndex` a zero. Questo è importante: se l'utente genera nuove flashcard, l'app deve ripartire dalla prima carta della nuova lista.

```
@override  
void didUpdateWidget(FlashcardCard oldWidget) {  
  super.didUpdateWidget(oldWidget);  
  if (oldWidget.flashcards != widget.flashcards) {  
    setState(() => currentIndex = 0);  
  }  
}
```

I pulsanti Indietro e Avanti modificano `currentIndex` solo se esiste una carta precedente o successiva. La carta viene ricreata con `ValueKey(currentIndex)`, così l'animazione fronte/retro riparte correttamente quando si cambia domanda.

15. Interfaccia grafica

L'interfaccia usa colori viola e sfondi chiari per mantenere coerenza visiva. StudyHeader comunica subito il nome dell'app e NavigationCards rende visibili gli strumenti disponibili.

```
class StudyHeader extends StatelessWidget {
  const StudyHeader({super.key});

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(
          Icons.school,
          size: 70,
          color: Colors.deepPurple[400],
        ),
        const SizedBox(width: 10),
        Text('StudySpace'),
      ],
    );
  }
}
```

Scelte UI

- card chiare con ombra leggera per separare le sezioni
- icone Material per rendere comprensibile la navigazione
- pulsanti viola coerenti con il tema generale
- TextField e dropdown per input semplici
- LinearProgressIndicator per rendere visivo l'avanzamento del timer

16. Persistenza e dati

Attualmente la persistenza riguarda soprattutto le flashcard generate, che vengono scritte in un file flashcards.json nella cartella documenti dell'app. Task e stato del timer sono invece mantenuti in memoria e si azzerano quando l'app viene riavviata.

```
final dir = await getApplicationDocumentsDirectory();
final file = File('${dir.path}/flashcards.json');
await file.writeAsString(content);
```

Limiti attuali

- le flashcard vengono salvate ma non ricaricate automaticamente all'avvio
- i task sono mantenuti in memoria e si perdono chiudendo l'app
- le sessioni Pomodoro completate non sono persistenti
- manca una schermata storico o archivio

Per una versione futura si potrebbe creare un piccolo service dati, usando file JSON o un database locale come SQLite. Questo separerebbe la logica di salvataggio dall'interfaccia.

17. Test e verifiche

La verifica del progetto può essere fatta con test manuali mirati. Essendo una app con molte interazioni UI, è importante provare ogni flusso da inizio a fine.

Test consigliati

- Avvio: eseguire flutter run e verificare che l'app apra StudySpace senza errori.
- Navigazione: premere Pomodoro, Tasks e Flashcards e controllare che la sezione visibile cambi correttamente.
- Pomodoro: testare Start, Stop, Reset e cambio dei minuti.
- Task: aggiungere, completare ed eliminare task con priorità diverse.
- API: avviare Ollama localmente e generare flashcard da un tema.
- Errore API: provare con servizio non attivo per verificare cosa migliorare nella gestione degli errori.

Comandi principali

```
flutter pub get  
flutter run
```

18. Miglioramenti futuri e conclusione

Miglioramenti futuri

- aggiungere gestione errori visibile quando la chiamata API fallisce
- salvare e ricaricare task e flashcard
- permettere all'utente di scegliere il numero di flashcard
- separare la logica API in un service dedicato
- aggiungere test automatici per parsing JSON e logica del timer
- migliorare layout responsive e accessibilità
- creare una schermata statistiche per sessioni Pomodoro e task completati

Conclusione

Lo sviluppo di StudySpace ha rappresentato molto più della semplice realizzazione di un'applicazione per il Modulo 306. Fin dall'inizio abbiamo voluto creare un progetto che rispondesse a un'esigenza reale e quotidiana: aiutare studenti e apprendisti a organizzare meglio il proprio studio, gestendo tempo, attività e ripasso all'interno di un unico strumento. Durante la realizzazione del progetto abbiamo avuto l'opportunità di mettere in pratica numerose competenze tecniche acquisite nel percorso formativo, approfondendo tecnologie come Flutter e Dart e confrontandoci con problematiche concrete tipiche dello sviluppo software. La gestione dello stato dell'applicazione, l'organizzazione del codice in componenti riutilizzabili, l'integrazione di servizi esterni tramite API e la manipolazione di dati JSON sono stati aspetti che ci hanno permesso di consolidare conoscenze teoriche attraverso un'applicazione pratica.

Uno degli elementi più interessanti è stato l'utilizzo dell'intelligenza artificiale per la generazione automatica di flashcard. Questa funzionalità ha aggiunto una componente innovativa al progetto e ci ha permesso di esplorare il collegamento tra applicazioni mobile e servizi AI, un tema sempre più rilevante nel panorama informatico attuale.

Oltre agli aspetti tecnici, il progetto ci ha insegnato l'importanza della pianificazione, della documentazione e della capacità di affrontare problemi imprevisti durante lo sviluppo. Molte delle soluzioni implementate sono nate da tentativi, verifiche e miglioramenti progressivi, mostrando come la realizzazione di un software sia un processo continuo di analisi e perfezionamento.

Guardando il risultato finale, riteniamo che StudySpace abbia raggiunto gli obiettivi prefissati. L'applicazione è funzionante, presenta una struttura chiara e offre strumenti utili per supportare lo studio. Allo stesso tempo, il progetto lascia spazio a numerose evoluzioni future, come una gestione più completa della persistenza dei dati, l'aggiunta di statistiche avanzate, il miglioramento dell'accessibilità e l'introduzione di nuove funzionalità basate sull'intelligenza artificiale.

In conclusione, StudySpace rappresenta per noi un'esperienza formativa significativa che ha unito creatività, progettazione e sviluppo software. Questo progetto ci ha permesso non solo di applicare le competenze richieste dal Modulo 306, ma anche di comprendere meglio il valore di un'applicazione costruita attorno a bisogni concreti degli utenti. Per questo motivo consideriamo il lavoro svolto un importante passo nel nostro percorso di crescita come sviluppatrici informatiche.

Fonti

1. Sviluppare applicazioni con Flutter – Vincenzo Giacchina
2. <https://dart.dev/blog/announcing-dart-3-11>
3. <https://dart.dev/libraries/async/async-await>
4. <https://dart.dev/get-dart>
5. <https://dart.dev/resources/language/number-representation>
6. <https://dart.dev/learn/tutorial/first-app>
7. https://dart.dev/tools/diagnostics/undefined_named_parameter
8. https://dart.dev/tools/diagnostics/extra_positional_arguments_could_be_named
9. <https://api.flutter.dev/flutter/material/AppBar-class.html>
10. <https://api.flutter.dev/flutter/widgets/Column-class.html>
11. <https://api.flutter.dev/flutter/widgets/IndexedStack-class.html>
12. <https://api.flutter.dev/flutter/widgets/MediaQuery-class.html>
13. <https://api.flutter.dev/flutter/widgets/NestedScrollView-class.html>
14. <https://api.flutter.dev/flutter/widgets/Padding-class.html>
15. <https://api.flutter.dev/flutter/material/Scaffold-class.html>
16. <https://api.flutter.dev/flutter/widgets/ScrollView-class.html>
17. <https://api.flutter.dev/flutter/dart-core/Stopwatch-class.html>
18. <https://api.flutter.dev/flutter/material/TabController-class.html>
19. <https://api.flutter.dev/flutter/material/TappableChipAttributes/onPressed.html>
20. <https://api.flutter.dev/flutter/material/TextField-class.html>
21. <https://api.flutter.dev/flutter/material/TextField/onChange.html>
22. <https://api.flutter.dev/flutter/painting/TextStyle-class.html>
23. <https://api.flutter.dev/flutter/dart-async/Timer-class.html>
24. <https://docs.flutter.dev>
25. <https://docs.flutter.dev/ui/adaptive-responsive/input>
26. <https://docs.flutter.dev/cookbook/networking/fetch-data>
27. <https://docs.flutter.dev/cookbook/navigation/navigation-basics>
28. <https://docs.flutter.dev/cookbook/forms/retrieve-input>
29. <https://docs.flutter.dev/search?q=style>
30. https://pub.dev/packages/flash_card
31. https://pub.dev/packages/flash_card/example
32. <https://stackoverflow.com/questions/45900387/multi-line-textfield-in-flutter>
33. <https://stackoverflow.com/questions/51326170/flutter-layout-row-column-share-width-expand-height>
34. <https://stackoverflow.com/questions/51767997/is-there-a-way-to-print-a-console-message-with-flutter>
35. <https://stackoverflow.com/questions/54610121/flutter-countdown-timer>
36. <https://stackoverflow.com/questions/64887521/how-to-find-the-length-of-the-json-in-flutter>
37. <https://stackoverflow.com/questions/68389586/how-to-extract-key-and-value-from-json-object-in-flutter>
38. <https://stackoverflow.com/questions/76329904/width-height-of-mediaquery-ofcontext-size-are-zero>